

实验：Louvain 算法

任务 1：完成调包方式执行 Louvain 算法

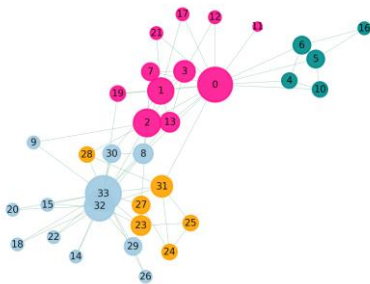
【实验任务】：完成 Notebook 脚本 louvain_1_api_todo.ipynb 中的【TODO】步骤：

▼ 【TODO】

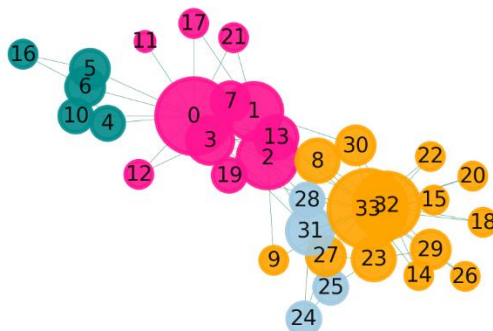
```
# -----  
# 2. 运行 Louvain 算法，获取每个节点所属的社区(分区结果)  
# 【TODO】 编写一行代码，调用community_louvain的best_partition函数执行Louvain算法  
#           请查看python-louvain 包的文档，学习best_partition函数的调用方法和返回值  
[9] ✓ 0.0s
```

添加一行代码后，确保脚本每个步骤均执行成功。

第 7 步可视化执行的结果如下面所示：



【粘贴截图】：请粘贴你的第 7 步可视化执行成功的结果图（注，Louvain 算法每次运行结果不同，故可视化结果每次执行也不相同）。



任务 2：不同密度社区对比

完成 Notebook 脚本 `louvain_1_todo.ipynb` 中的【TODO】步骤：

目的：通过 ER 随机图的连边概率差异，直观展示社区“紧密程度”变化

NetworkX：用于构建与分析复杂网络

Matplotlib：用于科学绘图与可视化

使用 NetworkX 包中的 `erdos_renyi_graph` 函数，生成 ER 随机图，具有 20 个节点，生成 10 次，每次图中的两点之间出现边的概论由 0.1 递增到 1

【实验任务】：完成 Notebook 脚本 `louvain_2_er_todo.ipynb` 中的【TODO】步骤：

TODO

生成不同边概率的ER随机图

要求：编写一行代码，调用nx的`erdos_renyi_graph`函数，生成 ER 随机图，要求图中节点数 $n = 20$ ，连边概率 $p = \text{rate}$

首先学习NetworkX包的`erdos_renyi_graph`函数文档

```
> ~
'''
随机生成网络说明
erdos_renyi_graph(n, p) 会生成一个包含 n 个节点的 ER 随机图：
    • n — 节点数量
    • p — 对任意两节点之间连边的概率
在本示例中 n = 20, p 从 0.1 递增到 1.0 (共 10 种)，依次观察网络由稀到密的变化
'''

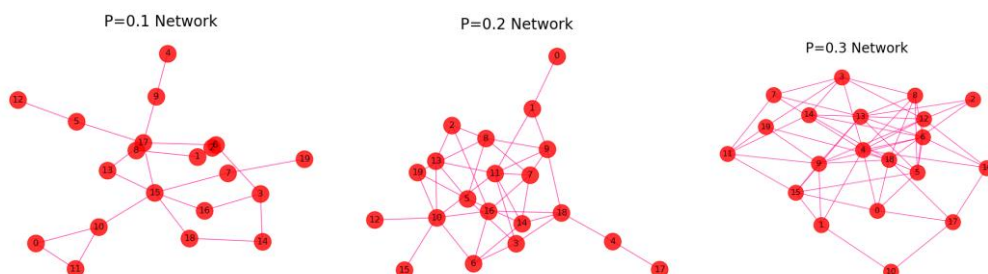
for i in range(1, 11):
    rate = i / 10
    print(rate)

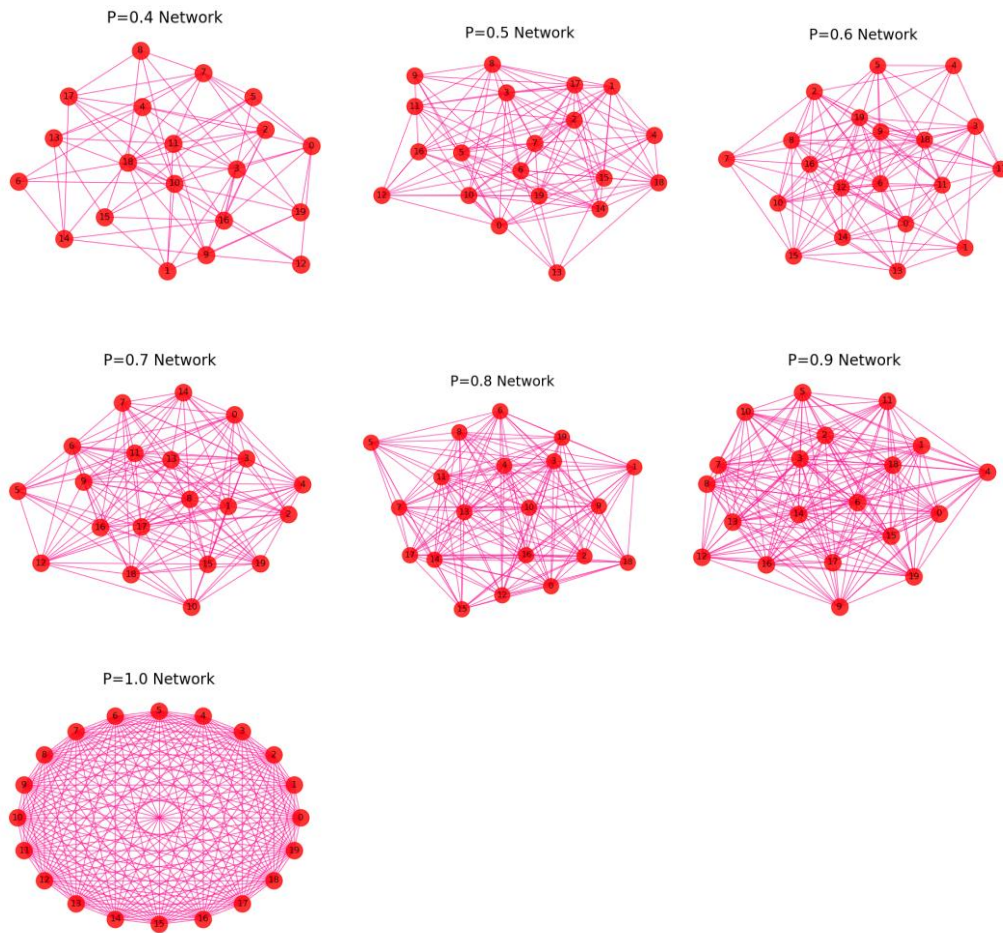
    # i = 1..10, 用于产生 10 个不同的连边概率
    # 计算当前循环对应的连边概率 p (0.1, 0.2, ..., 1.0)
    # 在控制台打印当前概率，便于跟踪进度

    # 【TODO】编写一行代码，调用nx的erdos_renyi_graph函数，生成 ER 随机图，要求图中节点数 n = 20，连边概率 p = rate
```

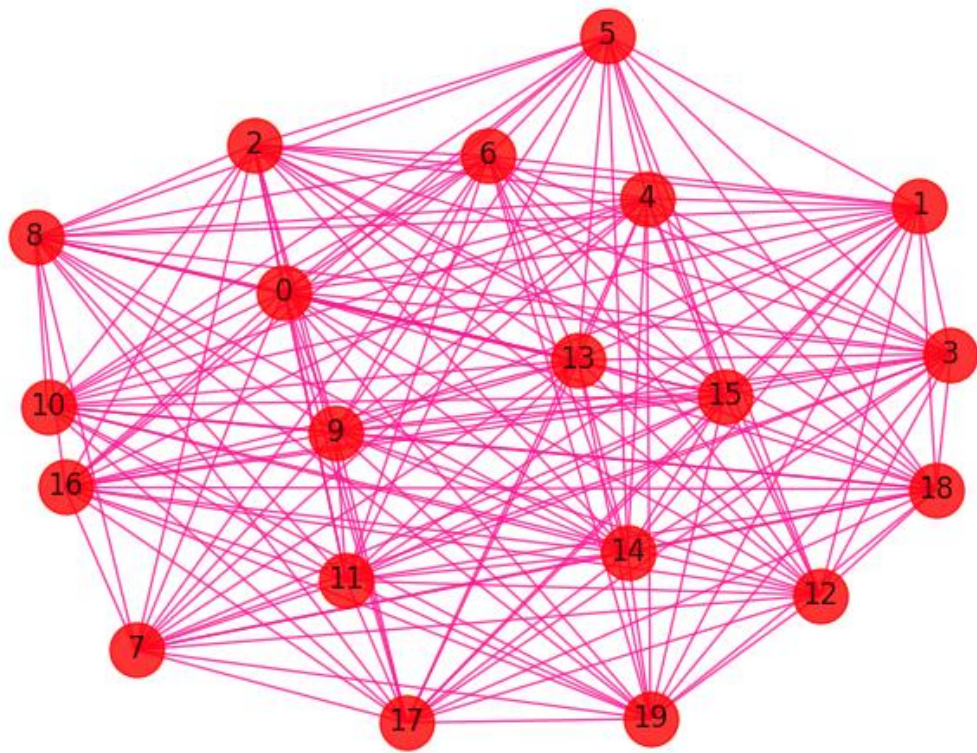
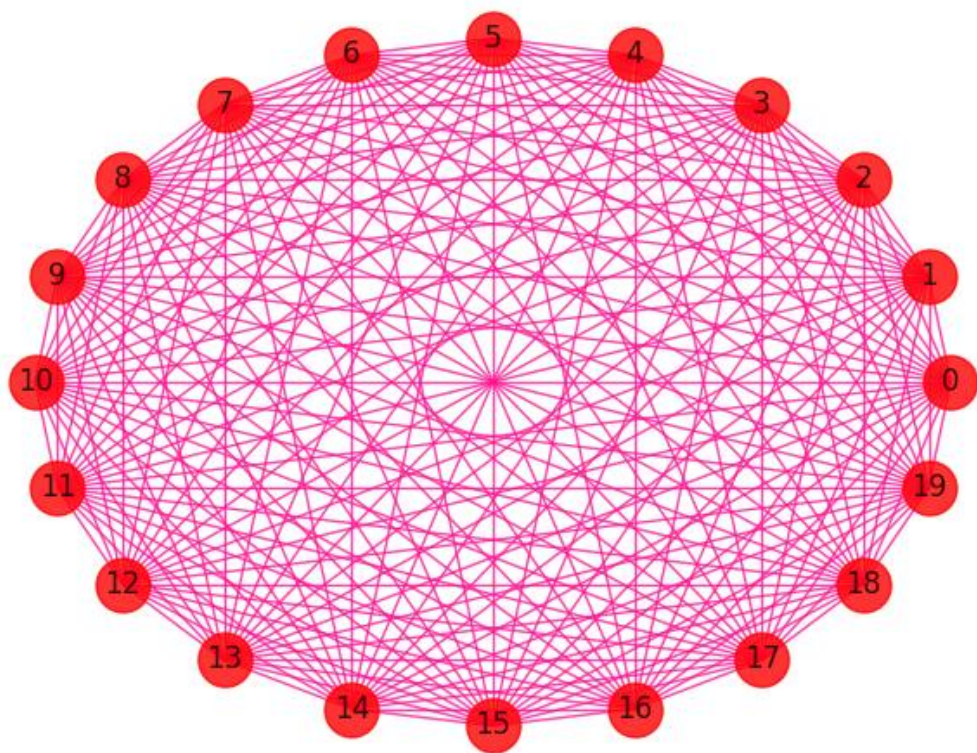
添加一行代码后，确保脚本每个步骤均执行成功。

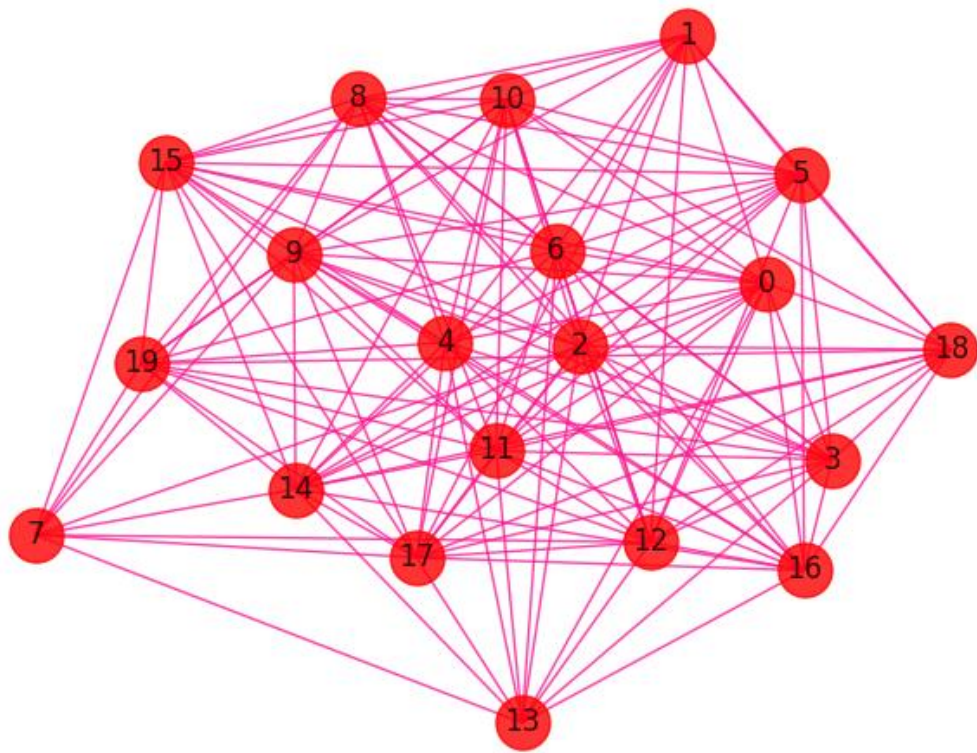
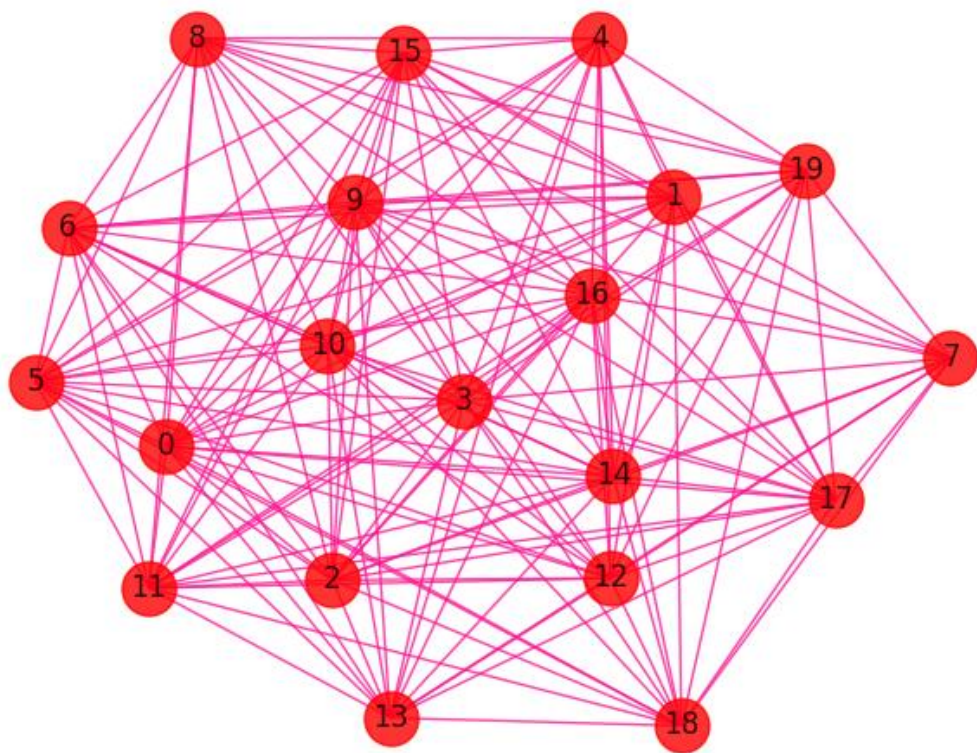
输出结果如下面所示：

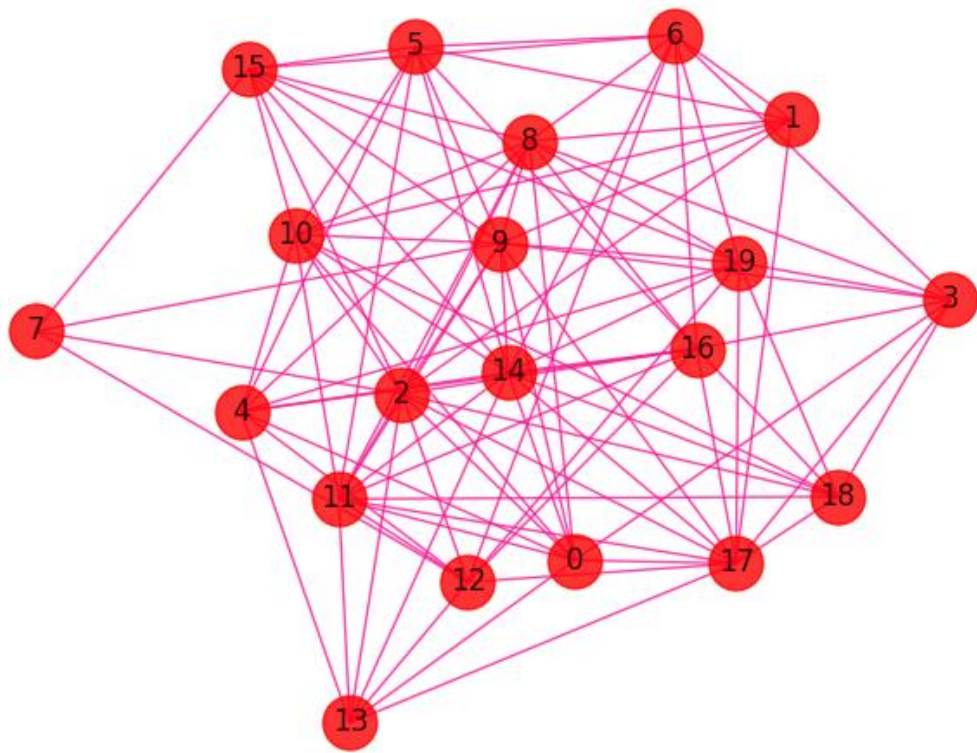
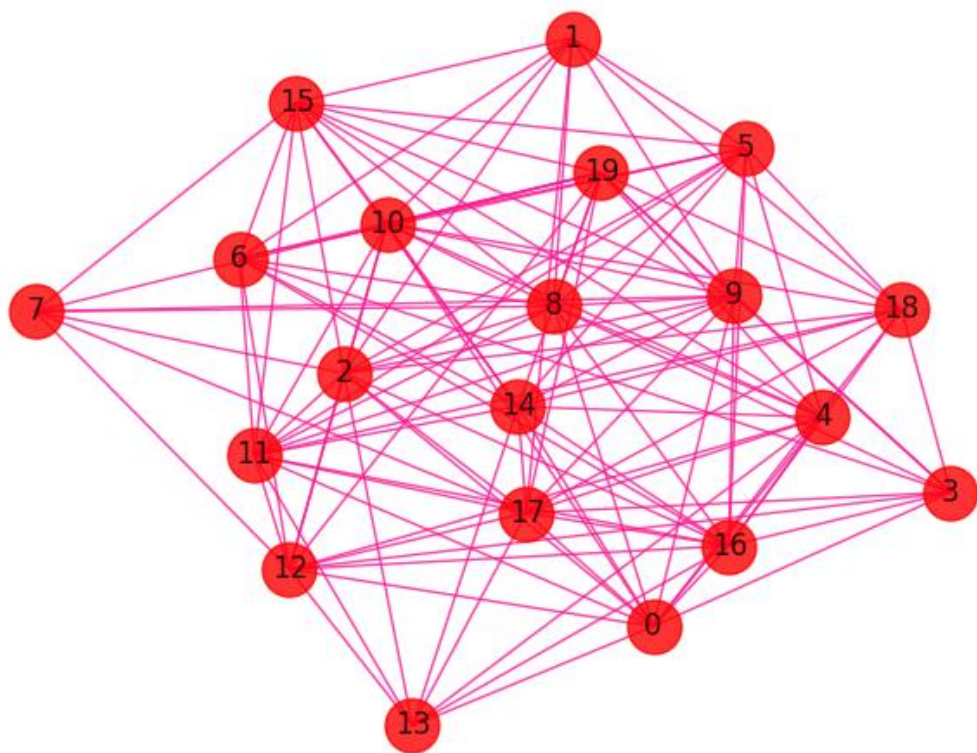


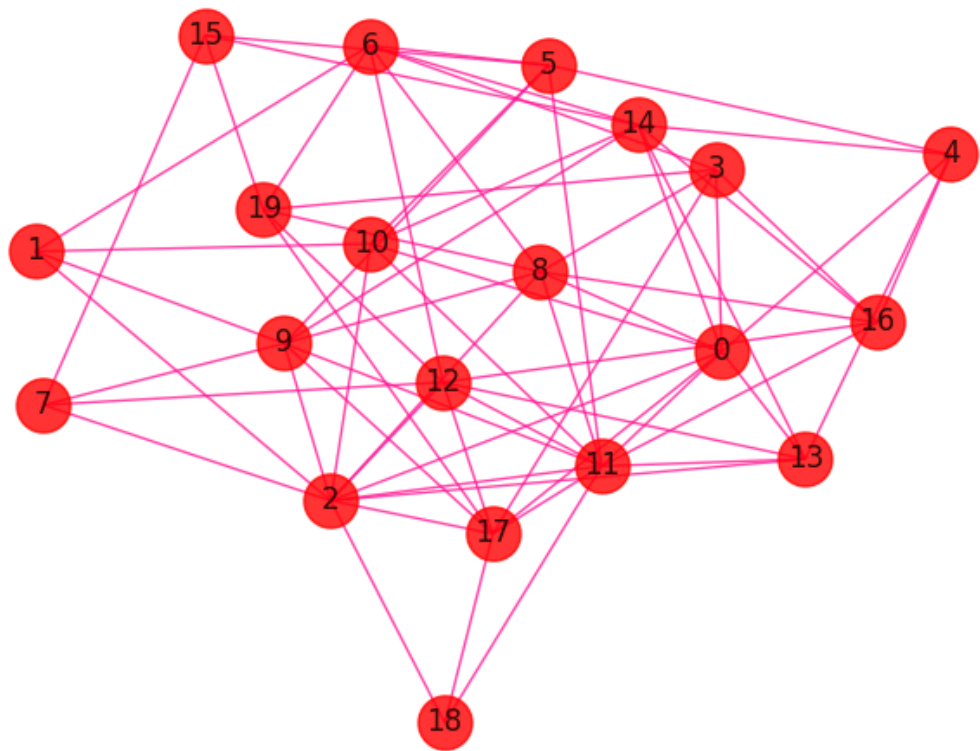
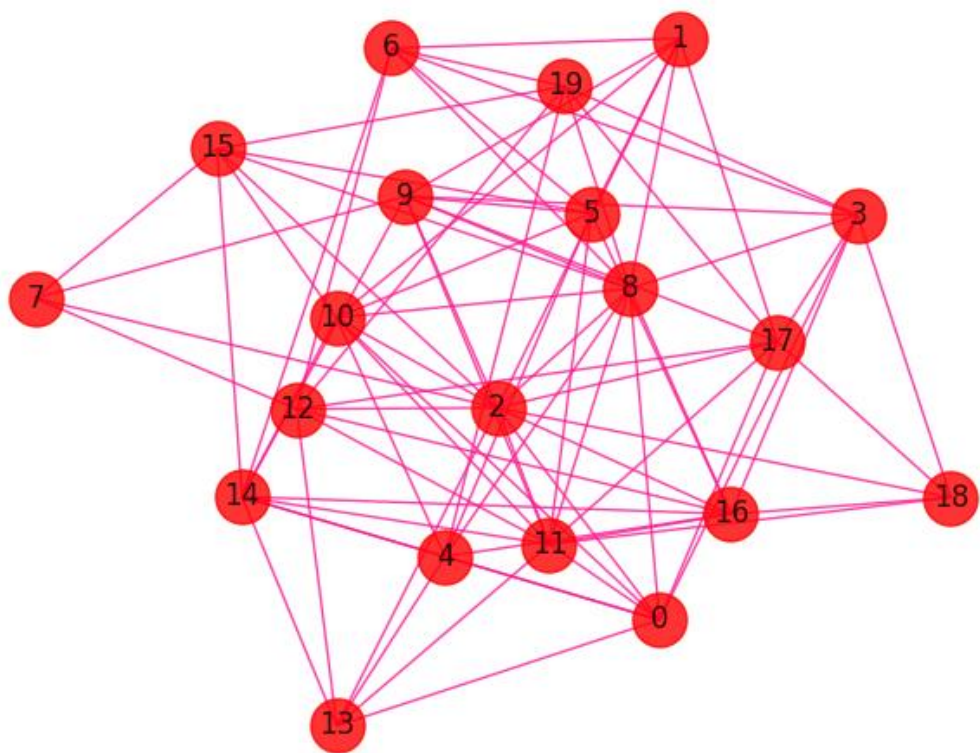


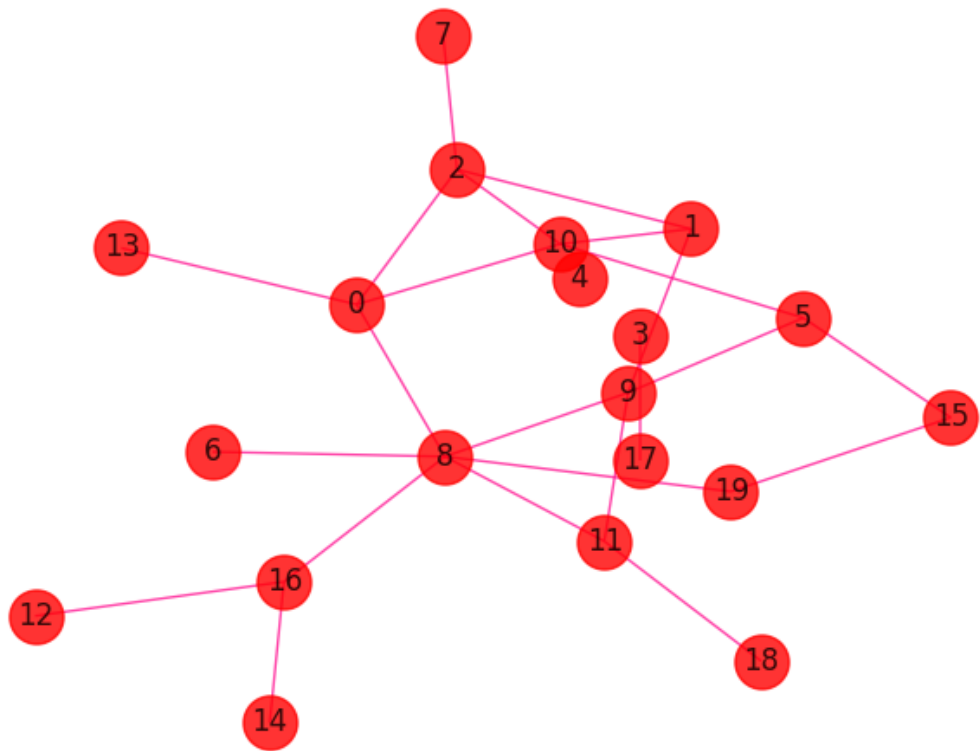
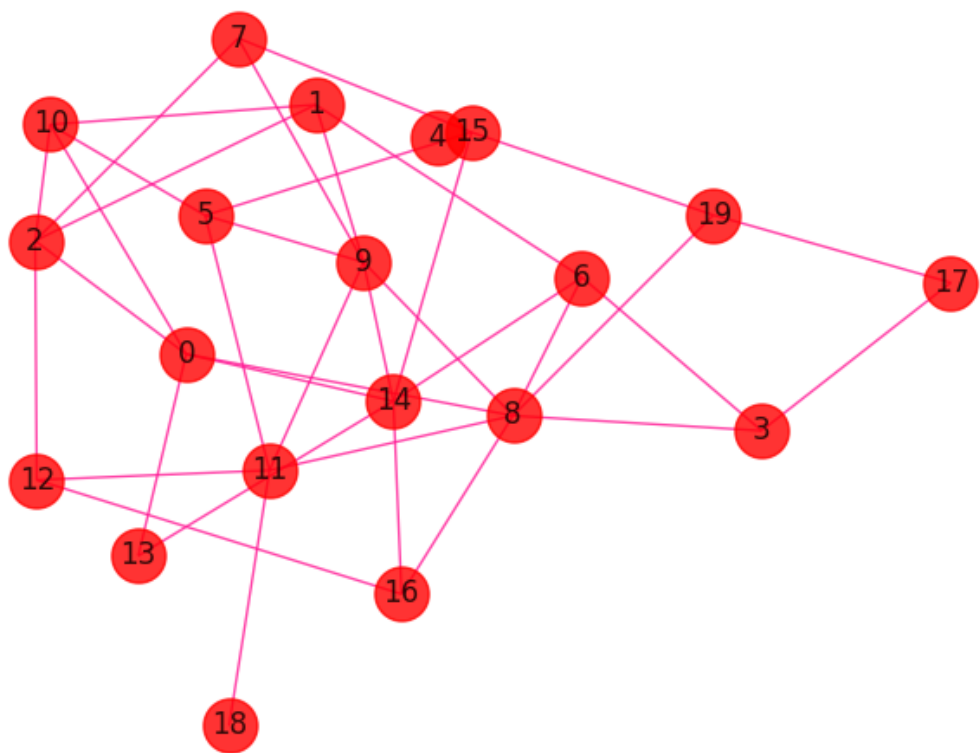
【粘贴截图】：请粘贴你的执行上述代码成功的可视化结果图（注，生成 ER 随机图每次运行结果不同，故可视化结果每次执行也不相同）。











任务 3：在 SNN 数据集上执行 Louvain 算法（调包方式）

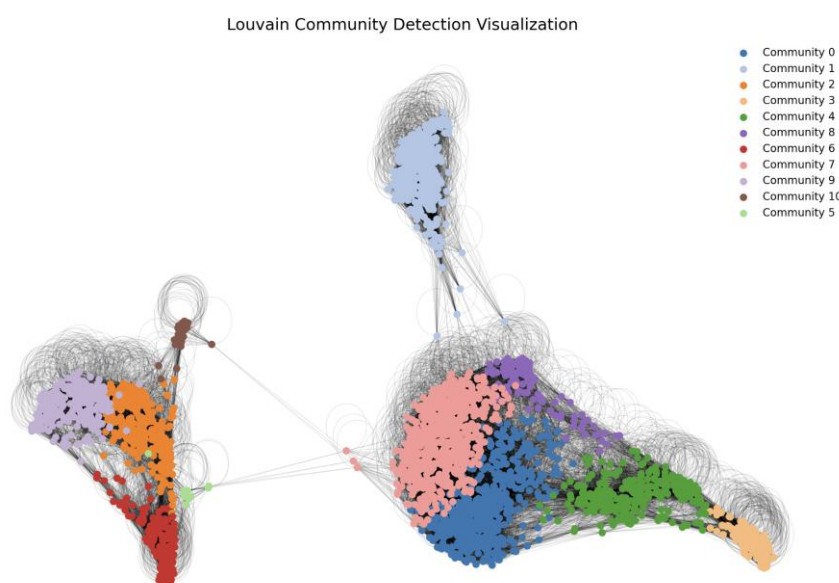
【实验任务】：执行 `louvain_1_api_large.py` 代码，成功执行结果如下：

```
{0, 1, 2, 3, 4, 5, 6, 7, 8, 9}
saved to file: backup/result-api-large.txt
Exec time: 1.52 seconds
```

执行 `louvain_1_api_large_vis.py` 代码，对输出结果文件进行可视化，执行结果如下：

```
rgument looks like a single numeric RGB or RGBA sequence, which
se its length matches with *x* & *y*. Please use the *color* ke
intend to specify the same RGB or RGBA value for all points.
node_collection = ax.scatter(
可视化结果已保存为 louvain_1_api_large_vis.png
```

生成的图像文件 `louvain_1_api_large_vis.png` 为：

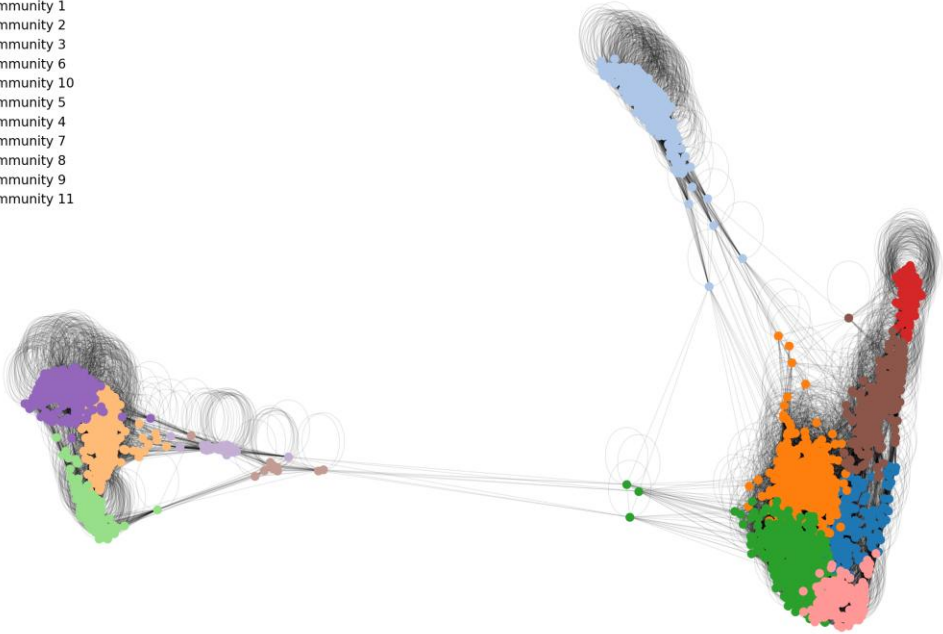


【粘贴截图】：请将上面可视化代码执行后生成的你的图像文件 `louvain_1_api_large_vis.png` 粘贴到此处，每次执行生成的结果文件可视化图像均不同。

```
{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11}
saved to file: backup/result-api-large.txt
Exec time: 1.33 seconds
```

Louvain Community Detection Visualization

- Community 0
- Community 1
- Community 2
- Community 3
- Community 6
- Community 10
- Community 5
- Community 4
- Community 7
- Community 8
- Community 9
- Community 11



任务 4: 执行 Louvain 算法的实现代码

【实验任务】：执行 `louvain_3_impl.py` 代码，成功执行结果如下：

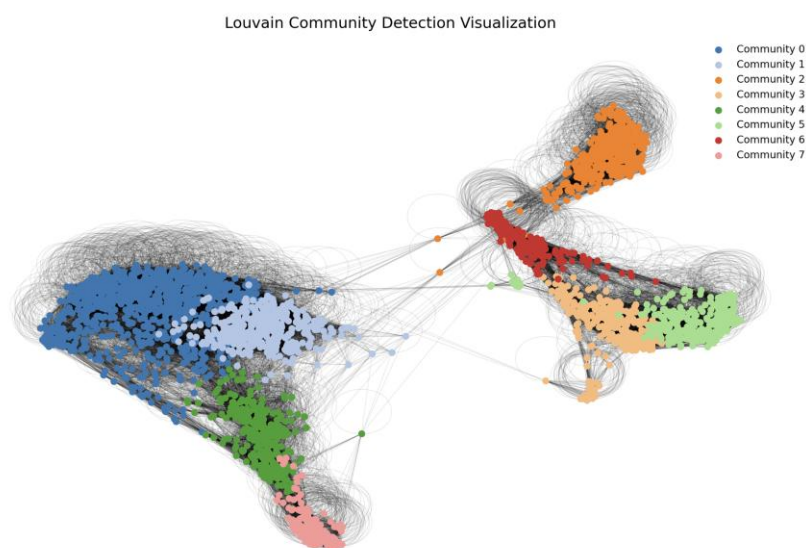
```
====>>> Total clusters: 8  
output_file: backup/result-impl.txt  
Exec time: 23.99 seconds
```

```
====>>> Total clusters: 10  
output_file: backup/result-impl.txt  
Exec time: 10.04 seconds
```

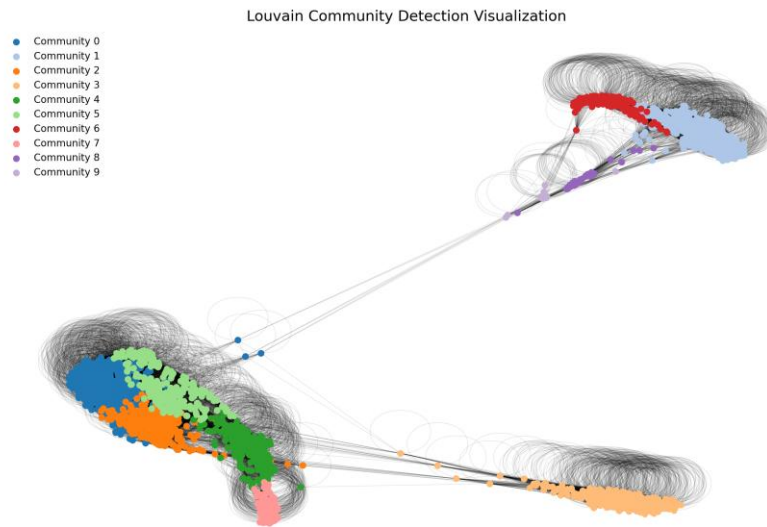
执行 `louvain_3_impl_vis.py` 代码，对输出结果文件进行可视化，执行结果如下：

可视化结果已保存为 `louvain_3_impl_vis.png`

生成的图像文件 `louvain_3_impl_vis.png` 为：



【粘贴截图】：请将上面可视化代码执行后生成的你的图像文件 `louvain_3_impl_vis.png` 粘贴到此处，每次执行生成的结果文件可视化图像均不同。



(选做，作对加分) 【源代码阅读】与 Louvain 算法原理对照，阅读 `louvain_3_impl.py` 源代码，并解决下面问题：

按照 Louvain 算法的模块度公式， $\Delta Q = \text{节点 } i \text{ 加入到社区 } C \text{ 中后的 } \Delta Q + \text{节点 } i \text{ 从社区 } D \text{ 中移出后的 } \Delta Q$ ，即

$$\Delta Q(D \rightarrow i \rightarrow C) = \Delta Q(D \rightarrow i) + \Delta Q(i \rightarrow C)$$

但在代码的 104 行 `delta_Q = delta_Q1 # + delta_Q2`

即只用了 `delta_Q1`，没有加上 `delta_Q2`。

去掉注释，加上 `delta_Q2`，观察程序的输出结果有何变化？

如果没有获得预期的社区结果，请你通过调试的方法，检查出 `delta_Q2` 计算中的问题，并修正，以期获得正常的社区结果。

【粘贴截图】：将修正的 `delta_Q2` 计算代码段截图粘贴到此处。

```
k_v_in_D = sum([w for k, w in self._G[v_vid].items()
if k in self._cid_vertices[v_cid]])
tot2 = sum([sum(self._G[k].values()) + self._vid_vertex[k]._kin
for k in self._cid_vertices[v_cid]])
delta_Q2 = k_v_in_D - ( (tot2 - k_v) * k_v ) / self._m
delta_Q = delta_Q1 - delta_Q2 cid_Q[w_cid] = delta_Q
```